Multimodal Crop Health Monitoring

EGN6217 - Applied DEEP Learning

Khang Phan - Spring 2026

github.com/itsmekhang/crop-health-monitoring

Problem Context and Project Summary

The agriculture industry has evolved rapidly in recent history and is now facing increasing pressure to efficiently use water, pesticides, and fertilizers, yet most farmers still rely on manual observation or delayed testing—leading to late disease detection, wasted resources, and reduced yields. This project builds a budget-friendly, modular AI system that detects crop diseases from leaf images using ResNet18 and assesses environmental risk from live weather values using gradient-boosted trees. The two models operate independently but are surfaced together through a Streamlit interface, giving farmers or researchers an interactive tool that combines visual and environmental insights in one interface. The system is designed to be scalable and extensible for future multimodal enhancements.

Dataset

PlantVillage (Image Classification)

- **Source:** Kaggle — emmarex/plantdisease
- **Type:** Labeled RGB leaf images, 224×224 px after resizing
- **Size:** ≈54,000 images within 38 disease/healthy classes
- **Access:** Kaggle API — `kaggle datasets download -d emmarex/plantdisease`
- **Preprocessing:** Resize to 224×224; normalize with ImageNet; augment with random flips, rotations (±15), and color jitter
- **Challenges:** Class imbalance; controlled lab lighting may not generalize well
- **Ethics:** Publicly available

USDA NASS QuickStats (Yield Baselines)

- **Source:** USDA National Agricultural Statistics Service — QuickStats API
- **Type:** API returning crop yield prices, and environmental values by state and year
- **Crops:** Tomatoes, Potatoes, Bell Peppers — matching the disease model's crop coverage
- **Access:** Free API key from quickstats.nass.usda.gov/api
- **Usage:** National average yield as the healthy-crop baseline for revenue loss estimation
- **Challenges:** US-specific data
- **Ethics:** Publicly funded government data; open access; no personal or sensitive information

Planned Architecture

Data Flow Diagram

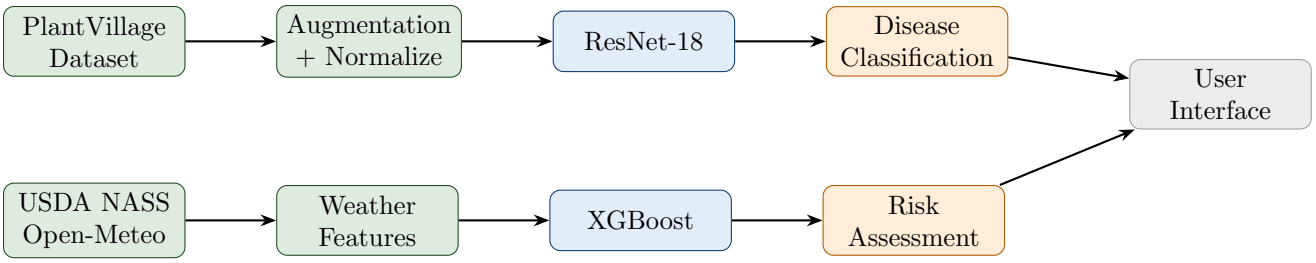

Image Model — Disease Classifier

- **Architecture:** ResNet-18 with ImageNet pretrained weights
- **Strategy:** Transfer learning
- **Fine-tuning:** After initial training, resume the last two ResNet blocks for domain adaptation
- **Loss:** CrossEntropyLoss
- **Optimizer:** Adam
- **Target metric:** Validation accuracy $\geq$85%

Tabular Model — Environmental Risk Scorer

- **Main model:** XGBoost Regressor (n_estimators=300, max_depth=5)
- **Features:** temperature, humidity, days since rain, ResNet confidence, disease class
- **Target:** Risk score in the range of $[0, 1]$ — derived from augmented disease-weather relationships
- **Weather source:** Open-Meteo API, fetched live by location at runtime
- **Yield baseline:** USDA NASS QuickStats API — national average yield per crop in lbs/acre
- **Achieved metric:** Validation $R^2 = 0.998$, Validation MAE = 0.008 on held-out split

Interface

The Streamlit user interface (`ui/app.py`) takes leaf image files, a location, and economic inputs (field size, crop price). Weather is fetched automatically from Open-Meteo using the location. The disease classifier analyzes the uploaded images; XGBoost scores risk from the disease result and live environmental conditions. Outputs return disease class, confidence, risk level, treatment plan, and estimated revenue loss.

User Interface Plan

Inputs and Outputs

Section	User Input	Output Displayed
Leaf Images	Upload one or more leaf photos	Disease class, confidence, top-5 predictions
Weather	City or region name	Live temp, humidity, days since rain
Economic Impact	Field size, crop price	Yield loss, revenue loss
Combined output per image		Risk score, risk level, treatment plan

Application Screenshots

Innovation and Anticipated Challenges

Innovation

This project is novel in its *practical multimodal design*: rather than forcing a single fused model which is difficult to train and debug, it trains two separate models and combine them together through one interface. This modular approach lowers the complexity for farmers to use AI tools without requiring technical knowledge. The combination of transfer learning on PlantVillage and gradient-boosted trees on environmental data includes both early disease detection and seasonal yield planning.

Implementation Timeline

Week	Focus	Expected Outcome
Apr 5 – Apr 12	Data collection, EDA, baseline models	Working data loaders; EDA plots in `setup.ipynb`; random forest baseline ($R^2 > 0.6$)
Apr 12 – Apr 19	ResNet-18 training; XGBoost tuning; metric tracking	Disease classifier ≥80% val accuracy; XGBoost $R^2 \geq 0.80$; `results/` populated
Apr 19 – Apr 26	Streamlit UI build; model integration; demo run	Fully interactive dashboard; end-to-end inference working on both functionalities
Apr 26 – Apr 29	Final report, README polish, repo cleanup	Completed report; tidy repository; presentation-ready prototype

Responsible AI Reflection

Fairness: The PlantVillage dataset was collected in consistent laboratory settings, which might not generalize well under a variety field conditions across different geographies, soil types, or lighting environments. A model trained only on this data could perform underwhelmingly for farmers in regions underrepresented in the dataset. Future work should evaluate performance separately by each crop class or conditions, and collect field images from real-world settings.

Transparency: The Streamlit interface shows top-5 class probabilities rather than a single hard prediction, giving users a sense of the model's desicion making. For the yield model, feature importance plots (saved to `results/`) make the model's reasoning inspectable.

Environmental impact: ResNet-18 is a relatively lightweight architecture. Training on a single GPU for 10 epochs on 54,000 images takes approximately 10-20 min. The XGBoost model is CPU-bound and trains in under a minute.

Misuse risk: Overconfident disease predictions could cause a farmer to implement the wrong mitigation technique or address the situation inappropriately. The UI is designed to communicate uncertainty to discourage overreliance on the model not being robust enough.

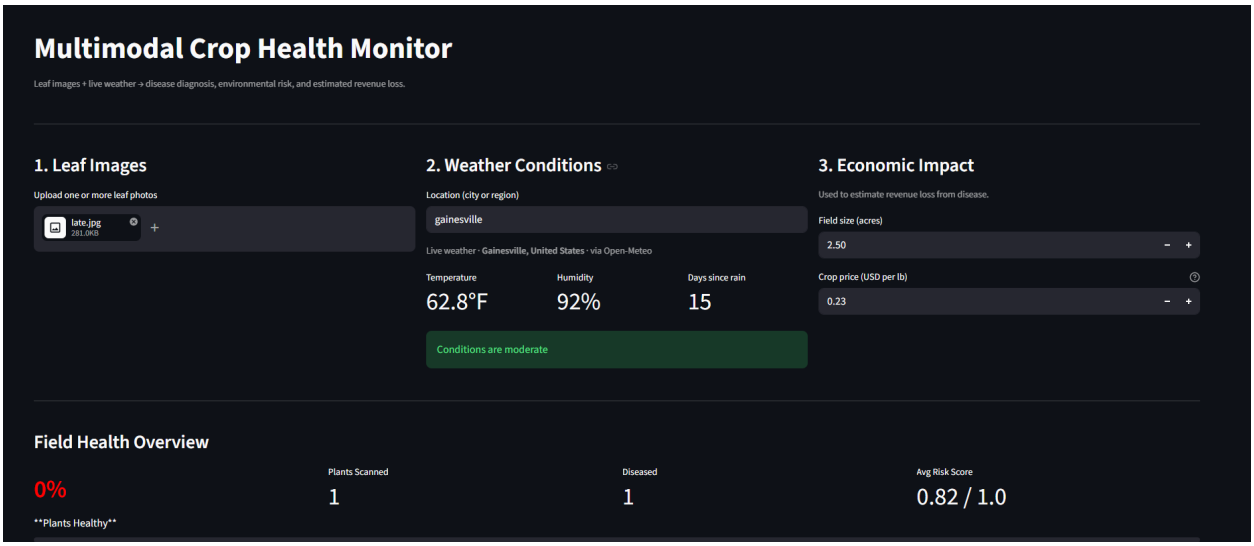

Figure 1: Dashboard input panel — leaf image upload, live weather via Open-Meteo, and economic impact inputs.

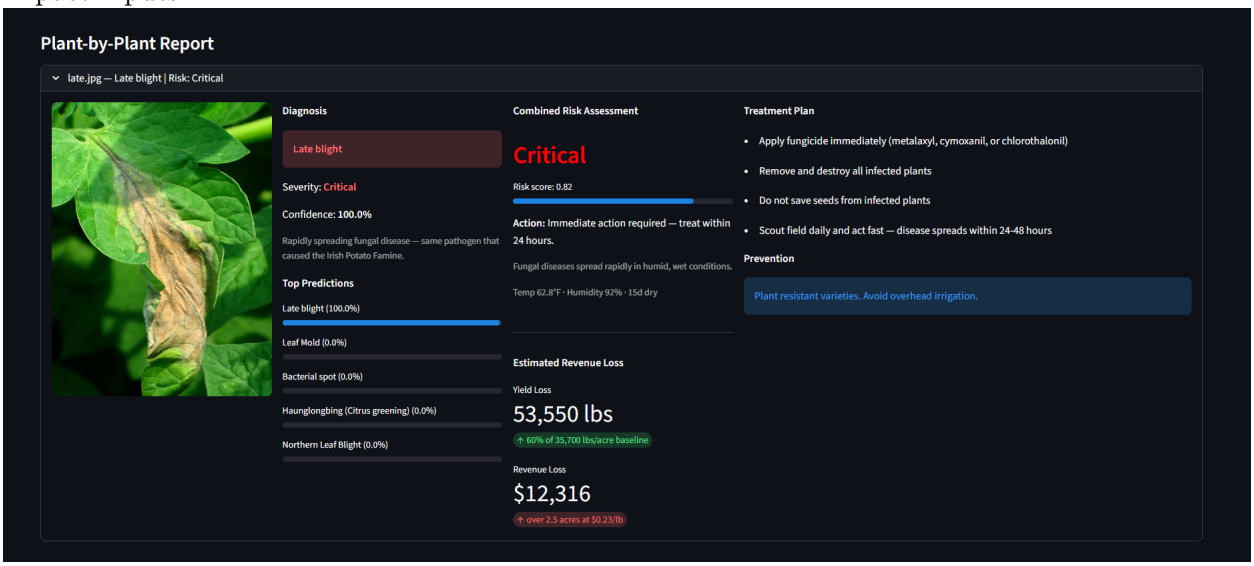

Figure 2: Plant-by-Plant Report — disease diagnosis, XGBoost risk assessment, treatment plan, and estimated revenue loss.